

CS-3510-B Problem Set 8

Elijah Martin McCauley

TOTAL POINTS

70 / 90

QUESTION 1

1 Problem 1 25 / 30

- **0 pts** Correct
- **30 pts** Completely Incorrect/ Missing

Proof of NP

- **5 pts** Incorrect/ Missing NP proof
- **3 pts** Major Error/Incomplete
- **1 pts** No runtime specified
- **2 pts** Minor error
- **20 pts** Reverse reduction provided (Party-

Invitation $\rightarrow$ NP-complete problem)

Reduction

- **15 pts** Incorrect/Missing Reduction
- ✓ - **5 pts** Minor Mistake (ex. small mistake in converting IS to Party-Invitation, Direction of the reduction not provided)
- **10 pts** Major Error (ex. reduction cannot be justified, correct problem to reduce from but incorrect reduction)

Bi-directional Proof

- **5 pts** Incorrect/ Missing $x \in A \implies f(x) \in B$
- **2 pts** Minor Error (i.e. Incomplete explanation)
- **4 pts** Major Error (i.e. forgot to prove it for all cases)

- **5 pts** Incorrect/ Missing $x \notin A \implies f(x) \notin B$
- **2 pts** Minor Error (i.e. Incomplete explanation)
- **4 pts** Major Error (i.e. forgot to prove it for all cases)

1 Adding a family set $F_v \subseteq V$ only containing the vertex v makes it impossible to add anyone to the party. these families are unnecessary.

QUESTION 2

2 Problem 2 30 / 30

- ✓ - **0 pts** Correct
- **30 pts** Completely Incorrect/ Missing

Proof of NP

- **5 pts** Incorrect/ Missing NP proof
- **3 pts** Major Error/Incomplete
- **0 pts** No runtime specified
- **2 pts** Minor error
- **20 pts** Reverse reduction provided (Average Sum $\rightarrow$ NP-complete problem)

Reduction

- **15 pts** Incorrect (Ex. Many edge cases)/ Missing Reduction
- **5 pts** Minor Mistake (ex. small mistake in converting Subset Sum to Average Sum, Direction of the reduction not provided)

- **10 pts** Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- **5 pts** Incorrect/ Missing $x \in A \implies f(x) \in B$

- **2 pts** Minor Error (i.e. Incomplete explanation)

- **4 pts** Major Error (i.e. forgot to prove it for all cases)

- **5 pts** Incorrect/ Missing $x \notin A \implies f(x) \notin B$

- **2 pts** Minor Error (i.e. Incomplete explanation)

- **4 pts** Major Error (i.e. forgot to prove it for all cases)

QUESTION 3

3 Problem 3 15 / 30

- **0 pts** Correct

- **30 pts** Completely Incorrect/ Missing

Proof of NP

- **5 pts** Incorrect/ Missing NP proof

- **3 pts** Major Error/Incomplete

- **0 pts** No runtime specified

- **2 pts** Minor error

- **20 pts** Reverse reduction provided (Clique or Independent Set $\implies$ NP-complete problem)

Reduction

✓ - **15 pts** *Incorrect/Missing Reduction*

- **5 pts** Minor Mistake (ex. small mistake in converting IS/Clique to Clique&IS, Direction of the reduction not provided)

- **10 pts** Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- **5 pts** Incorrect/ Missing $x \in A \implies f(x) \in B$

- **2 pts** Minor Error (i.e. Incomplete explanation)

- **4 pts** Major Error (i.e. forgot to prove it for all cases)

- **5 pts** Incorrect/ Missing $x \notin A \implies f(x) \notin B$

- **2 pts** Minor Error (i.e. Incomplete explanation)

- **4 pts** Major Error (i.e. forgot to prove it for all cases)

2 it will take at most $O(n^2)$

3 Your reduction can identify a IndSet, but given input graphs for situations in which a clique and IS is present and not present respectively, it would not return what we want

For example, your reduction doesn't ensure that when there is a IndSet, there is also a clique-and-IS. You need to add k disconnected vertices in this case to ensure this condition.

Problem 1

(30 points)

Show that the following **Party-Invitation** problem is NP-Complete.

Input: You are given a set of guests G of length m to be invited to a party, and a set of k individual family sets $F_1, F_2, \dots, F_k$ such that they are non-disjoint and $F_i \subseteq G$.

Output: A set Q of guests such that not all members of any one family F_i are invited to the party.

Note: Write the reduction in the format described on the front page of this document.

Solution: To prove that PartyInvitation is NP-Complete, we need to prove it is both in NP and NP-Hard. Let's start with NP. The witness is a set, Q , of guests. To prove that it is NP, we need to be able to verify the solution in polytime. We can do this by looping through the set of Q guests, and for each guest, loop through the families and check which it is in. If all the members of a family are in Q , then it is not a valid output. If we loop through all the guests without finding all the members of a family then it is valid. This runs in $\mathcal{O}(|Q| * k)$ because we need to loop through all Q guests and for each one loop through the families to see if it is in the set.

Now we need to prove it is NP-Hard by reducing a known NP-Complete problem to Party-Invitation in polytime. 1.) First we need to pick A , for this problem let's use Independent Set.

2.) We need to reduce Independent Set to Party-Invitation in polytime. Given an instance of the Independent Set problem with graph $G = (V, E)$ and integer k , we construct an instance of the Party-Invitation problem with guests $G = V$ and families $F_1, F_2, \dots, F_k$ as follows: For each vertex v in V , we create a family set F_v that contains only v . For each edge (u, v) in E , we add u and v to the same family set $F(u,v)$. The output of the Party-Invitation problem is a set Q of guests such that not all members of any one family F_i are invited to the party.

3.) We now to to prove that $x \in A \implies f(x) \in B$. For x to be $\in A$, there exists a subset V' of vertices such that $|V'| \geq k$ and no two vertices in V' are adjacent. Then, we can choose any subset Q of V' of length n to be invited to the party, and each family set $F(u,v)$ that contains two vertices in V' will have at most one member in Q . Since no two vertices in V' are adjacent, no family set $F(u,v)$ will have all its members in Q . Thus, Q is a valid solution to the Party-Invitation instance. Therefore if x is in A , then $f(x)$ is in B .

4.) Lastly, we must prove $x \notin A \implies f(x) \notin B$. For x to not be $\in A$, there is not a subset V' of length k which has no edges connecting the vertices of the subset. We can then not select a subset Q of V' of length n where no family will have all of its family invited because all subsets V' will have have an edge connecting two points, so some family set $F(u,v)$ that contains two vertices in V' will have at most one member in Q , but since V' has adjacent vertices, some family set $F(u,v)$ will have all its members in Q . Therefore, if x is not in A , then $f(x)$ is not in B .

5.) Since Independent Set was NP-complete, and $\text{IndependentSet} \leq_p \text{party-invitation}$, we can conclude that party-Invitation is NP-hard.

1 Problem 1 25 / 30

- 0 pts Correct
- 30 pts Completely Incorrect/ Missing

Proof of NP

- 5 pts Incorrect/ Missing NP proof
- 3 pts Major Error/Incomplete
- 1 pts No runtime specified
- 2 pts Minor error
- 20 pts Reverse reduction provided (Party-Invitation $\rightarrow$ NP-complete problem)

Reduction

- 15 pts Incorrect/Missing Reduction
- ✓ - 5 pts Minor Mistake (ex. small mistake in converting IS to Party-Invitation, Direction of the reduction not provided)
- 10 pts Major Error (ex. reduction cannot be justified, correct problem to reduce from but incorrect reduction)

Bi-directional Proof

- 5 pts Incorrect/ Missing $x \in A \implies f(x) \in B$
 - 2 pts Minor Error (i.e. Incomplete explanation)
 - 4 pts Major Error (i.e. forgot to prove it for all cases)
 - 5 pts Incorrect/ Missing $x \notin A \implies f(x) \notin B$
 - 2 pts Minor Error (i.e. Incomplete explanation)
 - 4 pts Major Error (i.e. forgot to prove it for all cases)
- 1 Adding a family set $F_v \coloneqq \{v\}$ only containing the vertex v makes it impossible to add anyone to the party. these families are unnecessary.

Problem 2

(30 points)

Show that the following **Average-Sum** problem is NP-Complete.

Input: a set S of natural numbers.

Output: A subset $S' \subset S$ such that the sum of the elements in the subset S' equals the average of elements in the set S i.e.

$$\sum_{s' \in S'} s' = \frac{1}{|S|} \sum_{s \in S} s$$

Note: Write the reduction in the format described on the front page of this document.

Solution: To prove that Average-Sum is NP-Complete, we need to prove it is both in NP and NP-Hard. Let's start with NP. The witness is a subset, S' , of S . To prove that it is NP, we need to be able to verify the solution in polytime. In order to check if the given subset adds up to the mean of the elements, we need to first calculate the mean which involves looping over the total set S and adding them together and dividing by the length of S . This would run in $\mathcal{O}(|S|)$. Then we add the numbers in the subset S' and see if it equals the calculated mean. This part would run in $\mathcal{O}(|S'|)$. So the overall verification runtime is $\mathcal{O}(|S| + |S'|)$ which is polytime.

Now we need to prove it is NP-Hard by reducing a known NP-Complete problem to Average-Sum in polytime. 1.) First we need to pick A , for this problem let's use SubsetSum.

2.) To reduce Subset Sum to Average Sum in polytime, we need to make the target t the average of the set after the reduction. This can be done by looping over every element in the input for Subset Sum, S , if it is greater than t , we can get rid of it, but if it is less than t , we will add a new element equal to $t + (t - \text{that element})$. This means that for each element, we are adding an element which together averages to t , so by doing this for all points, the average of the newly created set, $f(S)$ is t . This would run in poly time, because we are looping over the elements in S and either deleting them, which would take negligible time, or adding a corresponding element which would also take little time so the overall time of the reduction would be $\mathcal{O}(|S|)$.

3.) We now to prove that $x \in A \implies f(x) \in B$. For x to be $\in A$, some subset of the input needs to add to equal the goal t . Our reduction alters the input set, so that the goal t is now the average of the set, and we are not changing any values in the set that are less than t , so the subset S' that could add to t can now add to the mean (which is t). Therefore if a subset can be added to t , after the reduction it can be added to reach the new mean thus satisfying Average-Sum.

4.) Lastly, we must prove $x \notin A \implies f(x) \notin B$. For x to not be $\in A$, there is no subset of S that can add to the target t . After our reduction, the average of the set is t , and no new elements were added that are less than t , so if we could not reach t before the reduction we still cannot reach it and therefore cannot reach the mean. Thus if we cannot reach the target t , we cannot reach the mean after the reduction, so both will fail to return a subset.

5.) Since SubsetSum was NP-complete, and $\text{SubsetSum} \leq_p \text{Average-Sum}$, we can conclude that Average-Sum is NP-hard.

2 Problem 2 30 / 30

✓ - 0 pts Correct

- 30 pts Completely Incorrect/ Missing

Proof of NP

- 5 pts Incorrect/ Missing NP proof

- 3 pts Major Error/Incomplete

- 0 pts No runtime specified

- 2 pts Minor error

- 20 pts Reverse reduction provided (Average Sum $\rightarrow$ NP-complete problem)

Reduction

- 15 pts Incorrect (Ex. Many edge cases)/ Missing Reduction

- 5 pts Minor Mistake (ex. small mistake in converting Subset Sum to Average Sum, Direction of the reduction not provided)

- 10 pts Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- 5 pts Incorrect/ Missing $x \in A \implies f(x) \in B$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

- 5 pts Incorrect/ Missing $x \notin A \implies f(x) \notin B$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

Problem 3

(30 points)

Show that the following CLIQUE-&-INDEPENDENT-SET problem is NP-Complete.

Input: an undirected graph $G = (V, E)$ and an integer k .

Output: Returns both a k -clique and k -independent set in G i.e. a set $Q \subseteq V$ of k vertices that are fully connected, AND another set $R \subseteq V$ of k vertices with no edges between them. Sets R and Q might not be disjoint.

Note: Write the reduction in the format described on the front page of this document.

Solution: To prove that Clique and Independent set is NP-Complete, we need to prove it is both in NP and NP-Hard. Let's start with NP which means a solution can be verified in polytime. The witness is a set Q in V of k vertices and another set R in V of k vertices which might not be disjoint. We can verify the two of them separately. To verify that Q is fully connected, we can iterate through the edges in G and check that the source and destination of each edge is either both in Q or neither in Q . If they all edges are both in or neither in then it is a valid set. Looping through the edges to check would take $\mathcal{O}(2 + |E|)$. To verify if R has no edges between the vertices, we can iterate through the edges of G and confirm that the source and destination are not both in R in $\mathcal{O}(|E| + |V|)$ time. If none of the edges have both a source and destination in R , then the solution is valid.

Now we need to prove it is NP-Hard by reducing a known NP-Complete problem to Average-Sum in polytime. 1.) First we need to pick A , for this problem lets use IndSet.

2.) To reduce IndSet to Clique and IndSet, we take the indset input of a graph $G = (V, E)$, create a new graph $G' = (V, E_{\text{bar}})$ where $E_{\text{bar}} = \{(x, y) | (x, y) \notin E\}$. Basically, we make a graph in $\mathcal{O}(|E|)$ time where all the edges connecting vertices are the ones that were not in the first graph. We can make k equal the g which was passed into IndSet.

3.) We now to to prove that $x \in A \implies f(x) \in B$. For x to be $\in A$, there is a subset of vertices which has no edges connecting them and has a length $\geq g$. Because we kept the same graph as well as creating G' and $k = g$, we know that R will still work. If there is a solution x for IndSet, there is a solution for $f(x)$ because all the vertices in x have no edges connecting them, so when we constructed G' each pair of those vertices must have edges connecting them. Therefore if an input x works for Indset, $f(x)$ has a valid output for k -clique and k -independent, so Clique and IndSet has a valid output.

4.) Lastly, we must prove $x \notin A \implies f(x) \notin B$. For x to not be $\in A$, there is no subset of vertices of length g with no edges connecting them. If x is not a valid solution, then $f(x)$ will not have a valid solution because there cannot be an R of length k with no edges between them. Therefore if x is not in A , after reducing, $f(x)$ will not have a valid R , so Clique and Independent will fail.

5.) Since IndependentSet was NP-complete, and IndependentSet $\leq_p$ Clique and Independent, we can conclude that Clique and Independent is NP-hard.

3 Problem 3 15 / 30

- 0 pts Correct
- 30 pts Completely Incorrect/ Missing

Proof of NP

- 5 pts Incorrect/ Missing NP proof
- 3 pts Major Error/Incomplete
- 0 pts No runtime specified
- 2 pts Minor error
- 20 pts Reverse reduction provided (Clique or Independent Set $\rightarrow$ NP-complete problem)

Reduction

✓ - 15 pts *Incorrect/Missing Reduction*

- 5 pts Minor Mistake (ex. small mistake in converting IS/Clique to Clique&IS, Direction of the reduction not provided)
- 10 pts Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- 5 pts Incorrect/ Missing $x \in A \implies f(x) \in B$
- 2 pts Minor Error (i.e. Incomplete explanation)
- 4 pts Major Error (i.e. forgot to prove it for all cases)
- 5 pts Incorrect/ Missing $x \notin A \implies f(x) \notin B$
- 2 pts Minor Error (i.e. Incomplete explanation)
- 4 pts Major Error (i.e. forgot to prove it for all cases)

2 it will take atmost $O(n^2)$

3 Your reduction can identify a IndSet, but given input graphs for situations in which a clique and IS is present and not present respectively, it would not return what we want

For example, your reduction doesn't ensure that when there is a IndSet, there is also a clique-and-IS. You need to add k disconnected vertices in this case to ensure this condition.

Problem Set 8: NP-completeness II

Prof. Abraham Ladha

Due: 04/13/2023 11:59pm

- Please **TYPE** your solutions using **Latex** or any other software. **Handwritten solutions *won't* be accepted.**

Steps to write a reduction

To show that a problem is **NP**-complete, you need to show that the problem is both in **NP** and **NP**-hard. In a polynomial-time reduction, we have a *Problem B*, and we want to show that it is **NP**-complete. These are the steps:

- Demonstrate that *problem B* is in the class **NP**. This is a description of a procedure that verifies a candidate's solution. It needs to address the runtime. You should give a polytime verifier which takes as input a candidate problem, and a witness, and outputs true or false if the witness is a solution.
- Demonstrate that *problem B* is at least as hard as a problem previously proved to be **NP**-complete. Choose some **NP**-complete *A* and prove $A \leq_p B$. You need to show that *problem B* is **NP**-hard. This is done via polytime reduction from a known **NP**-complete *problem A* to the unknown *problem B* ($A \rightarrow B$) as follows:
 1. Choose *A*. You will have many **NP**-complete problems to pick from later on, and it is best to pick a similar one.
 2. Give your reduction f and argue that it runs in polynomial time.
 3. Prove that $x \in A \implies f(x) \in B$
 4. Prove that $x \notin A \implies f(x) \notin B$
 5. This is sufficient to show $x \in A \iff f(x) \in B$. Since *A* was **NP**-complete, and $A \leq_p B$, we can conclude that *B* is **NP**-hard. Note: You don't have to provide a formal proof. You can briefly explain in words both implications and why they hold.

The second bullet point above is prove that *problem B* is **NP**-hard which combined with the first bullet point yields the **NP**-complete proof.

Problem 4

(30 points)

Show that the following CYCLE-FREE problem is NP-Complete.

Input: a directed graph $G = (V, E)$ and an integer k .

Output: Returns a set $M \subseteq E$ of size at most k such that $|M| \leq k$ and a subgraph $G' = (V, E \setminus M)$ is cycle-free.

Note: Write the reduction in the format described on the front page of this document.

Solution: To prove that cyclefree is NP-Complete, we need to prove it is both in NP and NP-Hard. Let's start with NP. The witness is a set, M , of edges and a subgraph $G' = (V, E \setminus M)$. To prove that it is NP, we need to be able to verify the solution in polytime. To verify this, we can first run Explore on a point in the subgraph, G' . If it reaches all the vertices without reaching one a second time, it is cycle free. We then loop through the edges of G and check if they are either in M or G' . If both these conditions are true, then it is a valid solution. If explore reaches a vertex twice, or there is an edge of G not found in G' or M then it is not a valid solution.

Now we need to prove it is NP-Hard by reducing a known NP-Complete problem to Cycle Free in polytime. 1.) First we need to pick A , for this problem lets use CLIQUE.

2.) To reduce CLIQUE to Cycle-free in polytime, we take an instance of CLIQUE, which is a graph $G = (V, E)$ and an integer k , and we construct G' as follows: For every vertex u in V , add two vertices u and u' to V' . For every edge (u, w) in E , add an edge (u, w) and an edge (w, u') to E' . Set $k' = 2k$. We construct $G' = (V', E')$ from G by removing all vertices of the form u' . This can be done in polytime because we are simply looping over the vertices in V and then looping over the edges in E , and then looping over the vertices in V' to remove the u' 's. Therefore the runtime of the reduction is $\mathcal{O}(|E| + |V| + |V'|)$.

3.) We now to to prove that $x \in A \implies f(x) \in B$. For x to be $\in A$, G has a k -clique, which is a subset S of vertices in V with $|S| = k$ and where every pair of vertices in S is connected by an edge. Let $S = \{u : u \in S\}$. Note that since there is no edge between vertices of the form u and vertices of the form w' , no edge in E connects vertices in S and vertices not in S . Therefore, S is a set of k vertices in G that form a complete subgraph, and thus G' does not have a cycle.

4.) Finally, we prove $x \notin A \implies f(x) \notin B$. For x to not be $\in A$, there is not a subset of size k where all the vertices can connect to all the others. If there is not a subset that satisfies this, after our reduction we cannot create a graph G' which is cycle free and therefore if x is not in A , $f(x)$ cannot be in B .

5.) Since CLIQUE was NP-complete, and $\text{CLIQUE} \leq_p \text{CYCLE-FREE}$, we can conclude that CYCLE-FREE is NP-hard.

Problem 5 (*Optional, but highly recommended for the exam*)

Note: We won't grade this problem, but it's highly recommended that you solve the problem yourself.

Show that the following **4-coloring** problem is NP-Complete.

Input: An undirected graph $G = (V, E)$ and an integer $k = 4$.

Output: A 4-coloring of G if exists, else output NO